



CSN6214 OPERATING SYSTEM

TC1L TT4L

Group 8

NAME	STUDENT ID
LIM JIA XUAN	242UC244K9
ONG JING RU	242UC244P9
WAI YONG TONG	242UC244SP
WONG ZHEI LE	242UC244SQ

1. Game description and rules

This system implements a multiplayer board game played over a client-server network.

Each connected player is assigned a position on a 30-cell battlefield board. The game supports 3 to 5 players per round.

Players take turns in round-robin order. During a turn, a player rolls a dice that generates a value from 1 to 6. The player then moves forward based on the dice result.

There is trap cells located at positions 5, 15, and 25. When a player lands on a trap, the player moves back 3 cells. If the position becomes negative, it is reset to 0.

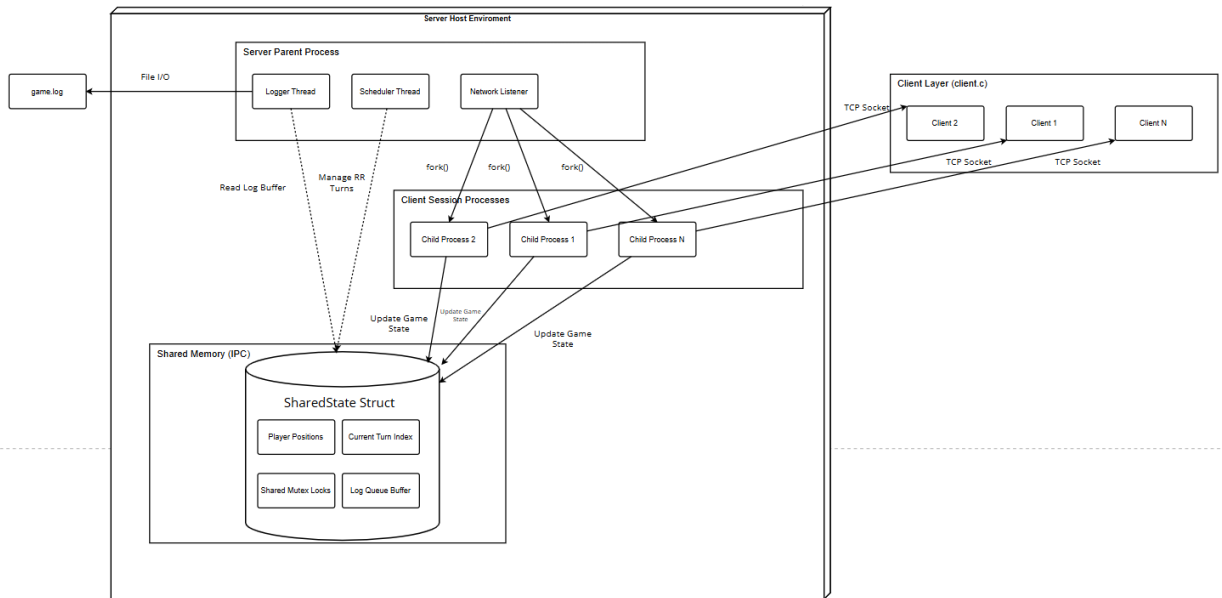
The first player who reaches position 30 wins the round. Once a winner is detected, the round ends and results are recorded. The system then returns to lobby mode and allows players to start a new round without restarting the server.

2. Deployment mode (TCP)

Mode Supported: TCP (Client–Server over network sockets)

- **Server:** uses TCP socket (socket, bind, listen, accept) to accept clients
- **Client:** connects using TCP (connect) with ./127.0.0.1 5555
- **Concurrency:** each client session runs in a child process (fork)
- **IPC inside server:** POSIX shared memory (shm_open, mmap) + process-shared mutex (PTHREAD_PROCESS_SHARED) is used to share game state across the forked processes

3. Hybrid architecture



The system uses a hybrid architecture combining multi-process, multi-threading, shared memory IPC, and TCP networking. The parent server process manages three main components: the network listener, scheduler thread, and logger thread. For each client connection, a child process is created to handle gameplay interaction.

All processes share a common memory region that stores player positions, turn index, and log buffers. Process-shared mutexes coordinate safe access to this data. Clients communicate with the server through TCP sockets, enabling real-time gameplay. Logging operations are handled asynchronously and written to persistent storage without interrupting game execution.

4. IPC mechanism

This system uses POSIX shared memory to share game state across processes.

The server main process creates or opens a shared memory object using `shm_open()`, sets its size with `ftruncate()`, and maps it into memory using `mmap(MAP_SHARED)`. After that, the server accepts TCP clients and `fork()` creates one child process per client. All child processes and the server threads read/write the same shared memory region.

To avoid race conditions, a process-shared mutex is placed inside the shared memory. The mutex is initialized with `pthread_mutexattr_setpshared(PTHREAD_PROCESS_SHARED)`. Any operation that reads or updates shared state (positions, turn, winner, player online status, etc.) is protected by `pthread_mutex_lock()` / `pthread_mutex_unlock()`.

In addition, a logger thread writes events into `game.log`, and scores persisted into `scores.txt` using snapshot saving to avoid long I/O holding the game lock.

Shared Memory Layout:

1. Player / Game Core State
 - num_players: number of online active players
 - positions[MAX_PLAYERS]: current position of each player on the board
 - current_turn: index of the player whose turn is active
 - game_active: whether a round is running (0/1)
 - winner: winner index, -1 means no winner yet
 - round_id: increases each new round (multi-round support)
2. Player Connection State
 - connected[MAX_PLAYERS]: whether the slot is connected
 - active[MAX_PLAYERS]: whether the player is active in the game
 - joined_players: total joined count (session tracking)
3. Control Flags (Round Lifecycle)
 - start_triggered: start request flag set by a client
 - turn_finished: set when a player finishes rolling so scheduler advances turn
 - game_just_started + game_start_time: short window to broadcast round start message
 - scores_dirty: set when scores change, triggers snapshot save
4. Synchronization Objects (Process-Shared)
 - pthread_mutex_t shm_mutex: protects all shared game state updates
 - pthread_mutex_t log_mutex: protects log ring buffer operations
5. Logger Ring Buffer (Shared Log Queue)
 - log_queue[20][128]: circular buffer storing log messages
 - log_head, log_tail: ring buffer indices

5. Synchronization strategy

The system uses mutex locks to coordinate shared data access between processes and threads. Since the server follows a hybrid architecture, multiple child processes and internal threads operate at the same time.

All game state data is stored in shared memory. A process-shared mutex is used to protect this memory. Before updating player positions, turns, or winner status, processes must acquire the mutex. This prevents simultaneous updates and ensures data consistency.

The scheduler thread also locks the same mutex when checking game progress and rotating turns. This avoids reading incomplete data while a player action is being processed. Logging operations are protected using a separate mutex to maintain correct log order.

Semaphores were not used because the system only requires mutual exclusion, not resource counting or signalling control.

6. Logger design

- **Thread Structure:**
 - Dedicated logger thread runs in server main process
 - Separate from gameplay execution
 - Prevents blocking during file write operations
- **Log Queue Design:**
 - Shared circular buffer used to store log messages
 - Fixed queue size
 - New events pushed by child processes and scheduler thread
 - Oldest entry overwritten if queue is full
- **Synchronization Safety:**
 - Queue protected using a mutex lock
 - Ensures atomic enqueue and dequeue operations
 - Prevents message interleaving or corruption
- **Log Writing:**
 - Logger thread continuously reads queue
 - Appends events into game.log
 - Flushes remaining logs during server shutdown

7. RR scheduler

- Thread Structure
 - Implemented as a dedicated scheduler thread
 - Runs in the main server process
 - Operates independently from child processes
- Shared Turn Control
 - Current turn stored in shared memory
 - Accessible by all child processes
 - Indicates which player can act
- Turn Coordination
 - Scheduler monitors turn completion status
 - Advances turn after player action finishes
 - Ensures only one player acts at a time
- Cross-Process Management
 - Child processes read shared turn value
 - Non-active players are restricted from acting
 - Ensures synchronized gameplay across sessions
- Inactive Player Handling
 - Scheduler checks active player list
 - Skips disconnected players automatically
- Synchronization Safety
 - Shared mutex protects turn updates
 - Prevents race conditions between processes
- Operational Outcome
 - Fair turn rotation
 - Centralized control of multi-process gameplay
 - Consistent shared game state

8. Persistence Strategy

- File Format (scores.txt)
 - Plain text file, one line per player
 - Fixed pattern so it can be parsed using fscanf
 - Example structure: Wins=2 Games=3 Rolls=46 Traps=7
- Loading Mechanism
 - Server loads scores during startup using load_scores()
 - If scores.txt does not exist, the server starts with all score values set to 0

- Loaded values are stored into shared memory arrays (player_wins[] and playerstats[])
- Saving Mechanism
 - The server writes scores using save_scores_snapshot()
 - The save is triggered when score-related values
 - A final save also happens during server shutdown to ensure persistence
- In-Memory Score Data
 - Scores are kept in shared memory (player_wins[], playerstats[])
 - All processes use the same shared copy, so scores remain consistent across child processes
- Synchronization and Data Protection
 - Updates to score data happen while holding the shared shm_mutex
 - This prevents two processes from modifying wins/stats at the same time
 - Saving to disk uses a snapshot approach:
 - Copy score values under shm_mutex
 - Release the mutex
 - Write the snapshot to scores.txt outside the lock
- This keeps gameplay responsive and avoids blocking game logic during file I/O

9. Multi-Game Handling

- Winner ends the round
 - When a player reaches TARGET_POS, the child process sets st->winner = player_idx.
 - The game is treated as finished once winner != -1.
- Scheduler detects game end
 - The scheduler thread keeps checking shared state.
 - If st->winner != -1, it logs the end, marks scores_dirty = 1, then waits 3 seconds (cooldown).
- Round reset (server-side)
 - After the cooldown, the scheduler resets the round state in shared memory:
 - st->game_active = 0
 - st->winner = -1
 - st->start_triggered = 0

- `st->turn_finished = 0`
 - `st->game_just_started = 0`
 - `st->game_start_time = 0`
 - All `st->positions[i] = 0`
- Players stay connected (`connected[]` / `active[]` are not cleared), so no server restart is needed.
- Start a new game
 - Any client can type start.
 - The child sets `st->start_triggered = 1`.
 - Scheduler sees: `!game_active && online >= MIN_PLAYERS && start_triggered == 1`, then initializes a new round:
 - `st->game_active = 1`
 - `st->round_id++`
 - reset winner, positions, `turn_finished`
 - selects the first active player as `current_turn`
 - sets a short broadcast window (`game_just_started` for ~2 seconds)
- Result
 - This creates a loop of Lobby → Start trigger → Round → Winner → Auto reset → Lobby, supporting multiple games without restarting the server.

10. Testing evidence

1. Server Startup

The server was compiled and executed successfully. It entered listening mode and waited for client connections.

```
live@minios:~$ cd /home/live/TT4L-G08/
live@minios:~/TT4L-G08$ ./server 5555
[SCORES] Loaded from scores.txt
Server listening on port 5555. Ctrl+C to stop.
[LOGGER] Started. Writing to game.log
```


2. Client Connection

Multiple clients connected to the server using TCP. Each client was assigned a unique player ID upon joining the game session.

```
live@minios:~/TT4L-G08$ ./client 127.0.0.1 5555
WELCOME Player 1
INFO Target=30
INFO Traps: 5, 15, 25 (move back 3)
INFO Commands: start | roll | board | state | q
```

```
live@minios:~$ cd /home/live/TT4L-G08/
live@minios:~/TT4L-G08$ ./client 127.0.0.1 5555
WELCOME Player 2
INFO Target=30
INFO Traps: 5, 15, 25 (move back 3)
INFO Commands: start | roll | board | state | q
```

```
live@minios:~$ cd /home/live/TT4L-G08/
live@minios:~/TT4L-G08$ ./client 127.0.0.1 5555
WELCOME Player 3
INFO Target=30
INFO Traps: 5, 15, 25 (move back 3)
INFO Commands: start | roll | board | state | q
```

3. Game Start Trigger

A player initiated the game using the start command. The scheduler detected sufficient players and initialized the round.

```
start
LOBBY: Start signaled.
STATE round=0 game_active=0 winner=0 turn=1 online=3 wins=4

server.c
=====
ROUND 1 STARTED
3 players racing to 30
Use 'roll' when it's your turn.
=====
```

4. Dice Roll and Movement

Players rolled dice during their turn. The system updated player positions based on the dice value.

```
roll
ACTION: You rolled 2. Position: 2

--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][!25 !]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][!15 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][P1 P2][ 3 ][ 4 ][!5 !]
-----
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
```

5. Trap Activation

If a player landed on trap cells (5, 15, 25), the system triggered a penalty and moved the player backward.

```
roll
ACTION: You rolled 5. TRAP! Back 3. Position: 2
server
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][!25 !]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][!15 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][P1 ][ 3 ][ 4 ][!5 !]
-----
STATE round=1 game_active=1 winner=0 turn=1 online=3 wins=4
```

6. Winner Detection

When a player reached the target position, the server declared the winner and broadcasted the result to all clients.

```
roll
ACTION: You rolled 3. Position: 30
server
--- CURRENT BATTLEFIELD ---
[P1 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][P3 ][!25 !]
[ 20 ][P2 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][!15 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][!5 !]
-----
STATE round=1 game_active=1 winner=1 turn=1 online=3 wins=5

=====
GAME OVER
Player 1 WINS
=====
```

7. Multi-Game Reset

After a short delay, the server reset the board and returned to lobby mode, allowing a new round to start without restarting the server.

```
Type 'start' for next round, or 'q' to quit.
[LOBBY] Ready. Type 'start' to begin a new round.
```


Player 2

```
live@minios:~/TT4L-608$ ./client 127.0.0.1 5555
WELCOME Player 2
CURRENT BATTLEFIELD ---
INFO Target=30
INFO Traps: 5, 15, 25 (move back 3)
INFO Commands: start | roll | board | state | q
=====
ROUND 1 STARTED!
3 players racing to 30
Use 'roll' when it's your turn.
=====
roll
ACTION: You rolled 3. TRAP! Back 3. Position: 2
ACTION: You rolled 2. Position: 2
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 6. Position: 8
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 6. Position: 14
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 2. Position: 16
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 3. Position: 19
```

```
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 2. Position: 21
ACTION: You rolled 3. TRAP! Back 3. Position: 22
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 5. Position: 26
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 1. Position: 27
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=0 turn=2 online=3 wins=0
roll
ACTION: You rolled 4. Position: 30
--- CURRENT BATTLEFIELD ---
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ]
=====
STATE round=1 game_active=1 winner=2 turn=2 online=3 wins=1
=====
GAME OVER
Player 2 WINS
Type 'start' for next round, or 'q' to quit.
=====
[LOBBY] Ready. Type 'start' to begin a new round.
```

Player 3

```
live@minios:/TT4L-608$ ./client 127.0.0.1 5555
WELCOME Player 3
INFO Target=30
INFO Traps: 5, 15, 25 (move back 3)
INFO Commands: start | roll | board | state | q
=====
ROUND 1 STARTED
3 players racing to 30
Use 'roll' when it's your turn. active=1 winner=0 turn=1 online=3
=====
roll
ACTION: You rolled 3. Position: 3
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ][ 25 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ][ 15 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ] 7 [ 6 ]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ] 4 [ 15 ]
[ 1 ][P1 P2][P3 ][ 4 ][15 !]
----- active=1 winner=0 turn=1 online=3
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 3. Position: 6
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ][ 25 ]
[ 21 ][ 22 ][ 23 ][ 24 ][ 25 ]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ][ 15 ]
[ 11 ][ 12 ][ 13 ][ 14 ][ 15 ]
[ 10 ][ 9 ][P2 ][P3 ]
[ 1 ][ 2 ][P1 ][ 4 ][15 !]
-----
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 6. Position: 12
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][125 !]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][P3 ][ 13 ][P2 ][115 !]
[ 10 ][P1 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][15 !]
-----
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 5. Position: 17
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][125 !]
[ 20 ][ 19 ][ 18 ][P3 ][P2 ]
[ 11 ][ 12 ][ 13 ][ 14 ][115 !]
[P1 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][15 !]
-----
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 2. Position: 19
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][ 23 ][ 24 ][125 !]
[ 20 ][P2 P3 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][P1 ][115 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][15 !]
----- active=1 winner=0 turn=1 online=3
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 4. Position: 23
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][ 26 ]
[ 21 ][ 22 ][P3 ][ 24 ][125 !]
[ 20 ][ 19 ][ 18 ][ 17 ][15 !]
[ 11 ][ 12 ][ 13 ][ 14 ][115 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][15 !]
----- active=1 winner=0 turn=1 online=3
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 1. Position: 24
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][ 27 ][P2 ]
[ 21 ][ 22 ][ 23 ][P3 ][125 !]
[P1 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][115 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][15 !]
-----
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
ACTION: You rolled 2. Position: 26
----- CURRENT BATTLEFIELD -----
[ 30 ][ 29 ][ 28 ][P2 ][P3 ]
[ 21 ][P1 ][ 23 ][ 24 ][125 !]
[ 20 ][ 19 ][ 18 ][ 17 ][ 16 ]
[ 11 ][ 12 ][ 13 ][ 14 ][115 !]
[ 10 ][ 9 ][ 8 ][ 7 ][ 6 ]
[ 1 ][ 2 ][ 3 ][ 4 ][15 !]
-----
STATE round=1 game_active=1 winner=0 turn=3 online=3 wins=0
roll
GAME OVER
Player 2 WINS
=====
Type 'start' for next round, or 'q' to quit.
> TIMELINE
[LOBBY] Ready. Type 'start' to begin a new round.
```

2. Logger Output (game.log)

Gameplay events were recorded by the logger thread and written into game.log. Captured events include connections, rolls, traps, turn changes, and winner announcements.

```
game.log
1  EVENT: Player 1 Disconnected
2  EVENT: Player 1 Connected
3  EVENT: Player 2 Connected
4  EVENT: Player 3 Connected
5  EVENT: Start signaled by a player
6  EVENT: Round 1 started (3 players)
7  EVENT: Player 1 rolled 5, hit TRAP, pos=2
8  EVENT: Turn advanced to Player 2
9  EVENT: Player 2 rolled 2 (pos=2)
10 EVENT: Turn advanced to Player 3
11 EVENT: Player 3 rolled 3 (pos=3)
12 EVENT: Turn advanced to Player 1
13 EVENT: Player 1 rolled 1 (pos=3)
14 EVENT: Turn advanced to Player 2
15 EVENT: Player 2 rolled 6 (pos=8)
16 EVENT: Turn advanced to Player 3
17 EVENT: Player 3 rolled 3 (pos=6)
18 EVENT: Turn advanced to Player 1
19 EVENT: Player 1 rolled 6 (pos=9)
20 EVENT: Turn advanced to Player 2
21 EVENT: Player 2 rolled 6 (pos=14)
22 EVENT: Turn advanced to Player 3
23 EVENT: Player 3 rolled 6 (pos=12)
24 EVENT: Turn advanced to Player 1
25 EVENT: Player 1 rolled 1 (pos=10)
26 EVENT: Turn advanced to Player 2
27 EVENT: Player 2 rolled 2 (pos=16)
28 EVENT: Turn advanced to Player 3
29 EVENT: Player 3 rolled 5 (pos=17)
30 EVENT: Turn advanced to Player 1
31 EVENT: Player 1 rolled 4 (pos=14)
32 EVENT: Turn advanced to Player 2
33 EVENT: Player 2 rolled 3 (pos=19)
34 EVENT: Turn advanced to Player 3
35 EVENT: Player 3 rolled 2 (pos=19)
36 EVENT: Turn advanced to Player 1
37 EVENT: Player 1 rolled 2 (pos=16)
38 EVENT: Turn advanced to Player 2
39 EVENT: Player 2 rolled 2 (pos=21)
40 EVENT: Turn advanced to Player 3
41 EVENT: Player 3 rolled 4 (pos=23)
42 EVENT: Turn advanced to Player 1
43 EVENT: Player 1 rolled 4 (pos=20)
44 EVENT: Turn advanced to Player 2
45 EVENT: Player 2 rolled 5 (pos=26)
46 EVENT: Turn advanced to Player 3
47 EVENT: Player 3 rolled 1 (pos=24)
48 EVENT: Turn advanced to Player 1
49 EVENT: Player 1 rolled 5, hit TRAP, pos=22
50 EVENT: Turn advanced to Player 2
51 EVENT: Player 2 rolled 1 (pos=27)
52 EVENT: Turn advanced to Player 3
53 EVENT: Player 3 rolled 2 (pos=26)
54 EVENT: Turn advanced to Player 1
55 EVENT: Player 1 rolled 3, hit TRAP, pos=22
56 EVENT: Turn advanced to Player 2
57 EVENT: Player 2 rolled 4 and WON
58 EVENT: Game ended. Auto reset in 3 sec
59 EVENT: Lobby ready. Waiting for next start
```

3. Persistent Storage (scores.txt)

Player statistics were stored persistently in scores.txt. Data remained available after server restarted.

```
scores.txt
1  Player 1: Wins=0 Games=1 Rolls=9 Traps=3
2  Player 2: Wins=1 Games=1 Rolls=8 Traps=0
3  Player 3: Wins=0 Games=1 Rolls=8 Traps=0
4  Player 4: Wins=0 Games=0 Rolls=0 Traps=0
5  Player 5: Wins=0 Games=0 Rolls=0 Traps=0
```

Name & ID	Primary Role	Key Components Developed / Implemented
LIM JIA XUAN 242UC244K9	Server Core & Process Management	[Server architecture, fork() process model, client session handling, SIGCHLD & zombie cleanup, shared memory setup, process-shared mutex]
ONG JING RU 242UC244P9	Round Robin Scheduler & Game State Control	[Scheduler thread, round-robin turn rotation, inactive player skip logic, start trigger handling, round reset, winner detection]
WAI YONG TONG 242UC244SP	Concurrent Logger & Event Recording	[Logger thread, circular log queue, mutex log protection, game.log writing, event recording, shutdown log flush]
WONG ZHEI LE 242UC244SQ	Client Interaction, Game Logic & Persistence	[TCP client program, command handling, dice & trap rules, win condition logic, scores.txt persistence, system testing]